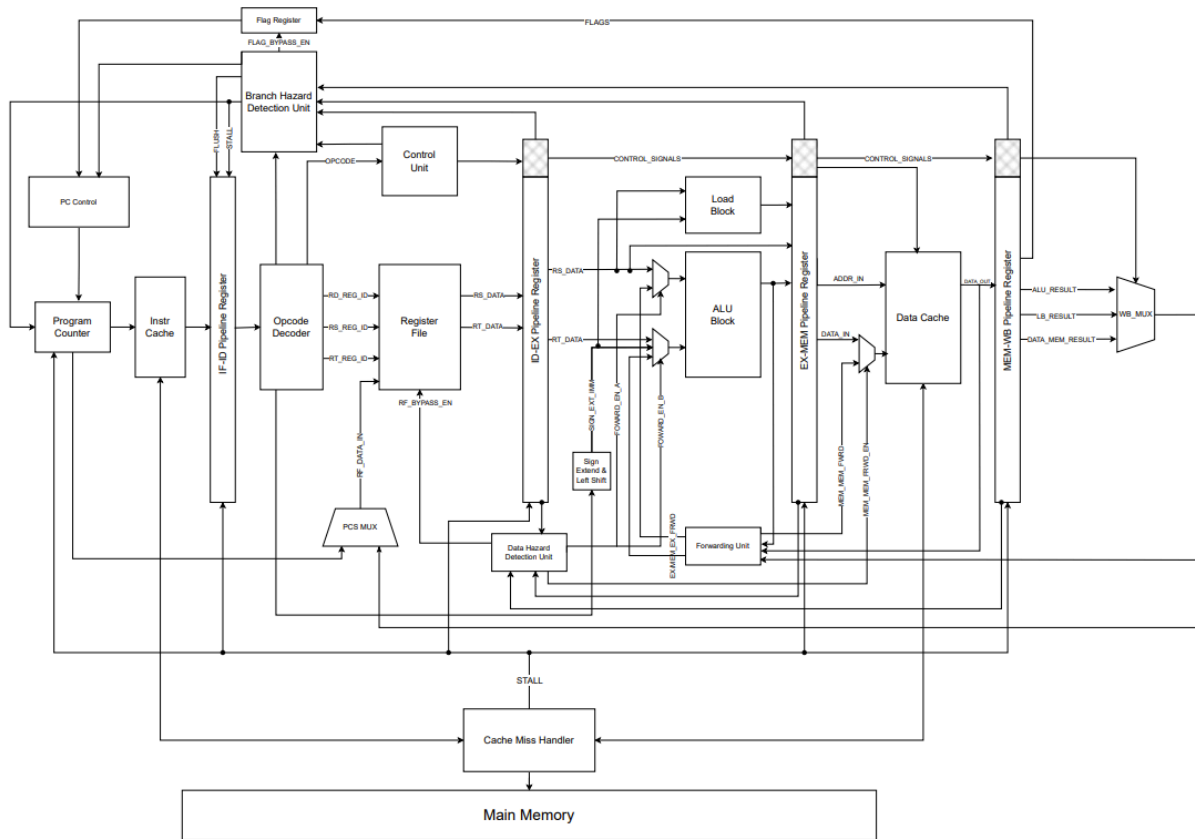


## 1) Project Overview



We developed a multi-cycle pipelined processor that fully supports the WISC-SP25 instruction set architecture (ISA). Our design incorporates two-way set-associative caches and a four-bank memory system shared between instruction and data accesses via a 2 way set associative cache. The project was organized into three incremental phases, each building on the previous to enhance both functionality and performance.

### Phase 1: Establishing the Single-Cycle Baseline

We began by implementing a single-cycle processor capable of executing all 16 instructions defined by the WISC-SP25 ISA, encompassing arithmetic, memory, and control operations. This processor featured separate instruction and data memories, with each instruction completing in one clock cycle. We emphasized modular design—developing reusable components such as the ALU, register file, program counter, and control logic—to simplify future pipelining efforts.

## Phase 2: Pipelining and Hazard Management

In the second phase, we restructured our design into a five-stage pipeline: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). To ensure correctness in the presence of data and control hazards, we implemented a hazard detection unit, data forwarding paths, and register bypassing. Enhanced branch logic allowed us to minimize unnecessary stalls, and overall throughput improved as instructions were now executed in parallel stages.

## Phase 3: Integrating Caches and Memory Hierarchy

Finally, we added two-way set-associative instruction and data caches (2KB each with 16-byte blocks), interfacing the I-cache with the IF stage and the D-cache with the MEM stage. Both caches followed a write-through, write-allocate policy. To handle cache misses, we implemented a pipelined finite state machine (FSM) capable of fetching 16-byte blocks in bursts (2 bytes per 4-cycle access). An arbiter was added to resolve conflicts when both caches attempted to access memory simultaneously. These enhancements completed our memory hierarchy and maintained pipeline efficiency even under memory contention.

## 2) Responsibility/Task Breakdown

As a two-person team, our workflow was highly collaborative and hands-on. Instead of dividing work strictly by modules or files, we approached each phase of the project together, frequently engaging in extended coding sessions where one member would drive (write code) while the other navigated (reviewed logic and design decisions in real time). Every component, from control logic to memory integration, was discussed, designed, and implemented with both of us fully present.

We did not rely heavily on modular file splitting to delegate responsibilities. Rather, we reviewed each requirement in the project documentation together, discussed our approach as a team, and took turns writing the implementation while maintaining constant feedback and discussion.

As such, our contributions were truly joint across all phases. Every line of code, design decision, and testing strategy was executed with full involvement from both team members.

| Phase   | Kushal Agrawal | Veda Sai Prakyath Duggirala | Notes               |
|---------|----------------|-----------------------------|---------------------|
| Phase 1 | 50%            | 50%                         | Fully collaborative |

|         |     |     |                                                                                  |
|---------|-----|-----|----------------------------------------------------------------------------------|
|         |     |     | design and implementation                                                        |
| Phase 2 | 50% | 50% | Jointly designed pipeline stages and involved forwarding, hazard and stall logic |
| Phase 3 | 50% | 50% | Worked together on cache integration and FSM handling                            |

### 3) Special Features

#### 3.1) Optimizations

While we did not implement any additional extra credit optimizations, we continuously improved and refined our design throughout the duration of the course. A major enhancement came during Phase 3, when we restructured our branch and control logic and created dedicated ALU modules for instructions that set flags. This modular approach improved clarity, reduced errors, and ensured consistent flag behavior.

We also modularized several components toward the end of the project, which made the codebase more manageable and significantly easier to debug and iterate on.

Each time we revisited the project, we identified opportunities to refine our implementation—resulting in more reliable and correct behavior across all stages.

#### 3.2) Major Bugs and Issues

##### Phase 1

- Our initial branch logic was flawed, causing an infinite loop in Test 3. This was due to improper flag handling—at the time, flags for execute instructions were still being set

directly within `cpu.v`. Updating the flag logic resolved the issue.

- The flags set by the ALU were being incorrectly interpreted in branch decisions, particularly for instructions like ADD that unintentionally affected condition flags.

## **Phase 2**

- We verified correct signal wiring across pipeline stages, but ran into a bug where the register file bypass logic was faulty—target registers were written with incorrect values when bypassing was triggered.
- We refined our forwarding and hazard detection logic through extensive experimentation with different stage signals, ultimately resolving the issue by aligning our approach with textbook designs and example cases.
- Integration became challenging due to the high number of interconnected signals across pipeline stages. Since most of our Phase 2 logic was contained in `cpu.v`, debugging signal dependencies proved complex and error-prone.
- A specific issue occurred where register file bypassing was incorrectly disabled when the register data value was 0, due to a misinterpretation of bypass conditions.
- We also discovered that our stall logic for branches was incorrect. The instruction we used as a placeholder (0x0000) behaved like an ADD and inadvertently set flags, which led to confusing branch behavior.

## **Phase 3**

- We restructured our cache data and metadata arrays to support 64 blocks, with separate arrays for instruction and data caches.
- A number of conceptual misunderstandings around cache operation—such as write policies and miss handling—required clarification through office hours and peer discussions.
- We modified the Mcell and Dcell modules to support reads independently of the write-enable signal, ensuring proper cache operation.
- Timing issues in the cache FSM were resolved by correcting how and when `instr_cache_miss` and `data_cache_miss` signals were asserted—previously, they caused

overlapping fetches and incorrect stalls.

- We fixed a bug where, on a write miss, the cache incorrectly fetched from the wrong block address due to mishandled offset logic.
- Our branch logic again failed due to flags being mishandled in the context of cached memory. We modularized the ALU operations to encapsulate flag setting more cleanly and ensure consistency.
- Several stalling issues related to cache misses, branches, and pipeline dependencies were identified and resolved.
- Finally, we modularized the cache system. Attempting to keep all logic in `cpu.v`—as we had done in Phase 2—became unsustainable due to growing complexity. Modularizing the cache code helped us debug and scale more effectively.

#### **4) Completeness**

Our design produced correct results for all test cases in phase1, phase2, and phase3 of the project.

#### **5) Testing**

For testing, we primarily relied on the `test.list` files provided with the project. While we occasionally modified our testbench to observe specific internal signals, our main strategy involved analyzing waveforms generated from simulation image files.

We used a combination of manual inspection and signal tracking to verify correctness. This included counting the total number of clock cycles per test case, measuring stalls due to branches or cache misses, and monitoring key outputs such as PC, DstReg, and DstData. Whenever discrepancies arose, we would trace the issue through the pipeline—checking enable signals, forwarding logic, stall control, and cache behavior as necessary.

Overall, our testing approach was waveform-driven: we made minimal changes to the provided test infrastructure, focused on accuracy through signal analysis, and debugged based on clear visual confirmation from simulation waveforms.

## 6) Results

We have included simulation results for Phase 3 based on the output log files and waveform analysis. These results reflect the correct behavior of our pipelined processor with cache integration, including proper handling of cache hits, misses, and stalls.

For Phases 1 and 2, since those stages were already graded and verified by the course staff, we are confident that the processor passed all corresponding test cases as intended.

|                              |                |
|------------------------------|----------------|
| <b>Phase 3 - Test Case 1</b> |                |
| <b>REG: 1 VALUE: 0x0051</b>  | <b>Correct</b> |
| <b>REG: 1 VALUE: 0x5151</b>  | <b>Correct</b> |
| <b>REG: 2 VALUE: 0x00b0</b>  | <b>Correct</b> |
| <b>REG: 2 VALUE: 0xa0b0</b>  | <b>Correct</b> |
| <b>REG: 3 VALUE: 0xf201</b>  | <b>Correct</b> |
| <b>REG: 4 VALUE: 0x7fff</b>  | <b>Correct</b> |
| <b>REG: 6 VALUE: 0x8dfe</b>  | <b>Correct</b> |
| <b>REG: 7 VALUE: 0x0000</b>  | <b>Correct</b> |
| <b>REG: 8 VALUE: 0xe37f</b>  | <b>Correct</b> |

|                                                                                                      |                |
|------------------------------------------------------------------------------------------------------|----------------|
| <b>REG: 9 VALUE: 0xdff8</b>                                                                          | <b>Correct</b> |
| <b>Sim Cycle Count: 44</b><br><br><b>Inst_count: 38 (Inflated due to insertion of xE000 PCS nop)</b> |                |

|                                          |                |
|------------------------------------------|----------------|
| <b>Phase 3 - Test Case 2</b>             |                |
| <b>REG: 1 VALUE: 0x0051</b>              | <b>Correct</b> |
| <b>REG: 1 VALUE: 0x5151</b>              | <b>Correct</b> |
| <b>REG: 2 VALUE: 0x00b0</b>              | <b>Correct</b> |
| <b>REG: 2 VALUE: 0x00b0</b>              | <b>Correct</b> |
| <b>REG: 3 VALUE: 0x0004</b>              | <b>Correct</b> |
| <b>REG: 3 VALUE: 0x0004</b>              | <b>Correct</b> |
| <b>STORE: ADDR: 0x00b4 VALUE: 0x5151</b> | <b>Correct</b> |
| <b>REG: 5 VALUE: 0x00b4</b>              | <b>Correct</b> |
| <b>LOAD: ADDR: 0x00b4 VALUE: 0x5151</b>  | <b>Correct</b> |

|                                                                                                      |                |
|------------------------------------------------------------------------------------------------------|----------------|
| <b>REG: 4 VALUE: 0x5151</b>                                                                          | <b>Correct</b> |
| <b>REG: 5 VALUE: 0x5101</b>                                                                          | <b>Correct</b> |
| <b>REG: 5 VALUE: 0x000e</b>                                                                          | <b>Correct</b> |
| <b>Sim Cycle Count: 54</b><br><br><b>Inst_count: 47 (Inflated due to insertion of xE000 PCS nop)</b> |                |

|                              |                |
|------------------------------|----------------|
| <b>Phase 3 - Test Case 3</b> |                |
| <b>REG: 1 VALUE: 0x0002</b>  | <b>Correct</b> |
| <b>REG: 1 VALUE: 0x0002</b>  | <b>Correct</b> |
| <b>REG: 2 VALUE: 0x0001</b>  | <b>Correct</b> |
| <b>REG: 2 VALUE: 0x0001</b>  | <b>Correct</b> |
| <b>REG: 6 VALUE: 0x0004</b>  | <b>Correct</b> |
| <b>REG: 6 VALUE: 0x0004</b>  | <b>Correct</b> |

|                                                                                                      |                |
|------------------------------------------------------------------------------------------------------|----------------|
| <b>REG: 1 VALUE: 0x0001</b>                                                                          | <b>Correct</b> |
| <b>REG: 5 VALUE: 0x0010</b>                                                                          | <b>Correct</b> |
| <b>REG: 2 VALUE: 0x0001</b>                                                                          | <b>Correct</b> |
| <b>REG: 2 VALUE: 0x0001</b>                                                                          | <b>Correct</b> |
| <b>REG: 6 VALUE: 0x0004</b>                                                                          | <b>Correct</b> |
| <b>REG: 6 VALUE: 0x0004</b>                                                                          | <b>Correct</b> |
| <b>REG: 1 VALUE: 0x0000</b>                                                                          | <b>Correct</b> |
| <b>REG: 5 VALUE: 0x0010</b>                                                                          | <b>Correct</b> |
| <b>REG: 4 VALUE: 0x0005</b>                                                                          | <b>Correct</b> |
| <b>Sim Cycle Count: 55</b><br><br><b>Inst_count: 45 (Inflated due to insertion of xE000 PCS nop)</b> |                |

|                                                  |
|--------------------------------------------------|
| <b>Phase 3 - Test Case 4</b>                     |
| <p>...</p> <p>(See Submitted Trace)</p>          |
| <b>REG: 1 VALUE: 0x00aa</b>                      |
| <b>REG: 1 VALUE: 0xaaaa (Correct PASS value)</b> |
| <b>Sim Cycle Count: 365</b>                      |

## 7) Extra Credit Synthesis

We made a serious effort to synthesize our final implementation and spent the entirety of Friday attempting to get it working. We submitted the best version of our own synthesis script and also tested the official ECE 552 TA-provided script. Unfortunately, neither approach resulted in a successful synthesis. In both cases, the process would hang indefinitely without producing output, and we were unable to determine whether the root cause was related to our code structure or the synthesis environment.

To document our efforts, we have included screenshots showing the synthesis terminal and the point at which the process appeared to stall alongside our script files.